

8种主流Agent框架

大模型Agents开发框架如今已经百花齐放，层出不穷。本文将盘点8种主流LLM Agents开发框架，并介绍如何在每种框架中集成当下备受关注的MCP Server，让Agents系统更方便的接入外部工具。

一、Open AI Agents SDK

【框架简介】



OpenAI Agents SDK是OpenAI官方推出的轻量级Agent开发框架，旨在方便开发者构建多Agent协作的智能体系统。该SDK源于OpenAI内部实验项目Swarm，并在近期正式推出生产版本。OpenAI Agents SDK的特点是：简单易用、轻量级、专注在最小集功能，并支持转交（Handoffs）、护栏（Guardrails）等很有特点的功能。

【集成MCP】

以下代码演示了如何将OpenAI Agent实例连接到一个搜索的MCP Server，并将其中的工具集成Agent中：

代码块

```
1  import asyncio, os
2  from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel,
   RunConfig
3  from agents.mcp import MCPServerStdio
4
5  async def main():
6      # 1. 创建MCP Server实例
7      search_server = MCPServerStdio(
8          params={
9              "command": "npx",
10             "args": ["-y", "@mcptools/mcp-tavily"],
11             "env": {**os.environ}
12         }
13     )
14     await search_server.connect()
15
16     # 2. 创建Agent并集成MCP Server
```

```

17     agent = Agent(
18         name="助手Agent",
19         instructions="你是一个具有网页搜索能力的助手，必要时使用搜索工具获取信息。",
20         mcp_servers=[search_server], # 将MCP Server列表传入Agent
21     )
22
23     # 3. 运行Agent，让其自动决定何时调用搜索工具
24     result = await Runner.run(agent, "Llama4.0发布了
25     吗?", run_config=RunConfig(tracing_disabled=True))
26     print(result.final_output)
27
28     await search_server.cleanup()
29
30 if __name__ == "__main__":
31     asyncio.run(main())

```

有趣的是，在使用远程MCP Server时，Agents SDK提供了自动缓存工具列表的选项（通过设置 `cache_tools_list=True`）。如果需要手动使缓存失效，可以调用MCP Server实例上的 `invalidate_tools_cache()` 方法。

二、LangGraph

【框架简介】



LangGraph来自著名的LangChain，是一个用于构建Agentic Workflow的强大框架，它将任务过程建模为有状态的Graph结构，从而可以实现更复杂和结构化的交互。在该框架内集成MCP Server可以在工作流程的各个阶段更精确地控制何时以及如何调用外部工具，从而实现复杂的Agentic系统。

LangGraph的特点是功能强大，你可以使用Prebuilt的接口快速创建Agent，也可以使用Graph定义复杂的Agentic工作流与多Agent系统；缺点是略显复杂。

【集成MCP】

将前面的示例修改为LangGraph+MCP Server的代码实现：

代码块

```

1  import asyncio, os
2  from langchain_mcp_adapters.client import MultiServerMCPClient

```

```
3
4 from langchain_core.messages import SystemMessage, HumanMessage
5 from langchain_openai import ChatOpenAI
6 from dotenv import load_dotenv
7 from langgraph.prebuilt import create_react_agent
8
9 # 加载环境变量
10 load_dotenv()
11
12 # 定义大语言模型
13 model = ChatOpenAI(model="gpt-4o-mini")
14
15 # 定义并运行agent
16 async def run_agent():
17     # 定义MCP服务器，用于访问Tavily搜索工具
18     async with MultiServerMCPClient(
19         {
20             "tavily": {
21                 "command": "npx",
22                 "args": ["-y", "@mcp-tools/mcp-tavily"],
23                 "env": {**os.environ} # 传递环境变量给MCP工具
24             }
25         }
26     ) as client:
27
28         # 创建ReAct风格的agent
29         agent = create_react_agent(model, client.get_tools())
30
31         # 定义系统消息，指导如何使用工具
32         system_message = SystemMessage(content=(
33             "你是一个具有网页搜索能力的助手，必要时使用搜索工具获取信息。"
34         ))
35
36         # 处理查询
37         agent_response = await agent.ainvoke({"messages": [system_message,
38             HumanMessage(content="Llama4.0发布了吗? ")])
39
40         # 返回agent的回答
41         return agent_response["messages"][-1].content
42
43 # 运行agent
44 if __name__ == "__main__":
45     response = asyncio.run(run_agent())
46     print("\n最终回答:", response)
```

注意这里使用MultiServerMCPClient可以灵活的支持多个MCP Server的同时连接，对于单个Server场景，你也可以借助load_mcp_tools方法直接从MCP SDK的session中导入Tools（无需MultiServerMCPClient）。

三、LlamaIndex

【框架简介】



LlamaIndex最初是一个专注于构建基于外部数据的LLM应用程序的框架，其独特之处在于构建以数据为中心的LLM应用的能力，特别是复杂的企业级RAG应用。但随着LlamaIndex Workflows与AgentWorkflow功能的推出，LlamaIndex也发展为一个更全能的专注于企业级RAG+Agent系统的开发框架。特点是功能强大、预置大量RAG应用优化模块；事件驱动的Workflows在Agent开发上比LangGraph更简单。

【集成MCP】

LlamaIndex目前也支持与MCP Server集成，快速导入Tools使用：

代码块

```
1  from llama_index.tools.mcp import McpToolSpec, BasicMCPClient
2  import asyncio
3  from llama_index.llms.openai import OpenAI
4  from llama_index.core.agent import ReActAgent
5  import os
6
7  llm = OpenAI(model="gpt-4o-mini")
8
9  async def main():
10
11      mcp_client = BasicMCPClient("npx", ["-y", "@mcptools/mcp-tavily"], env=
12      {**os.environ})
13      mcp_tool = McpToolSpec(client=mcp_client)
14      tools = await mcp_tool.to_tool_list_async()
15
16      agent = ReActAgent.from_tools(
17          tools,
18          llm=llm,
19          verbose=True,
20          system_prompt="你是一个具有网页搜索能力的助手，必要时使用搜索工具获取信息。"
```

```

20     )
21
22     response = await agent.aquery("Llama4.0发布了么? ")
23     print(response)
24
25 if __name__ == "__main__":
26     asyncio.run(main())

```

如果你的MCP Server是远程SSE模式运行，只需要更换BasicMCPClient初始化时的参数，将命令及参数（如npx）更换为url即可。

四、AutoGen 0.4+

【框架简介】



AutoGen是微软开发的一个框架，用于构建具有多Agent对话的下一代企业级AI应用。其独特之处在于专注于通过多个Agent之间的协调交互来实现协作和解决复杂任务，在最新的AutoGen0.4中，微软进行了颠覆性的架构修改，特别是开放了AutoGen-Core这一更底层的API层，可用于构建更底层与细粒度控制的分布式多Agent系统。其特点是功能强大，支持分布式多Agent，可根据需要选择不同层次的API使用；缺点是较复杂。

【集成MCP】

在Autogen 0.4的扩展中提供了MCP集成的组件，演示如下（代码有省略）：

代码块

```

1  from autogen_ext.tools.mcp import StdioServerParams, mcp_server_tools
2  ...
3
4  async def get_mcp_tools():
5      server_params = StdioServerParams(
6          command="npx",
7          args = [
8              "-y",
9              "@mcptools/mcp-tavily",
10             ],env={**os.environ}
11         )
12     tools = await mcp_server_tools(server_params)
13     return tools

```

```

14  ...
15
16
17  class ToolUseAgent(RoutedAgent):
18  ...
19
20
21  async def main():
22      """主函数，设置并运行agent系统"""
23      # 创建单线程agent运行时
24      runtime = SingleThreadedAgentRuntime()
25
26      mcp_tools = await get_mcp_tools()
27      tools = [*mcp_tools]
28
29      # 注册agent类型
30      await ToolUseAgent.register(runtime, "my_agent", lambda:
ToolUseAgent(tools))
31  ...
32
33      message = Message('Llama4.0发布了么? ')
34      response = await runtime.send_message(message, AgentId("my_agent",
"default"))

```

如果需连接远程MCP Server，请使用SseServerParams组件，并使用url参数初始化。

五、Pydantic AI

【框架简介】



Pydantic AI来自于著名的Pydantic库开发者，是一个将Pydantic与LLM集成的Agents开发框架。其独特之处在于专注于在AI应用中利用Pydantic的类型验证、序列化与结构化输出等功能。Pydantic AI的特点是天然的结构化输出与强类型验证，且简洁易用，与其他框架也有良好的集成，可以结合使用。

【集成MCP】

使用Pydantic AI集成MCP Server中的工具非常简单（与OpenAI Agents SDK非常类似），只需要简单的提供Server配置即可：

```

1  from pydantic_ai import Agent
代码块
2  from pydantic_ai.mcp import MCPServerStdio
3  import os
4
5  server = MCPServerStdio(
6      'npx',
7      ["-y", "@mcptools/mcp-tavily"],
8      env={**os.environ}
9  )
10
11  agent = Agent(
12      name="助手Agent",
13      system_prompt="你是一个具有网页搜索能力的助手，必要时使用搜索工具获取信息。",
14      model='openai:gpt-4o-mini',
15      mcp_servers=[server])
16
17  async def main():
18      asyncwith agent.run_mcp_servers():
19          result = await agent.run('"Llama4.0发布了吗?')
20          print(result.data)
21
22  if __name__ == "__main__":
23      import asyncio
24      asyncio.run(main())

```

如果需要使用SSE远程MCP，将Server组件更改为MCPServerHTTP即可。

六、SmolAgents

【框架简介】



Smloagents是大名鼎鼎的Hugging Face开发的一个轻量级Agent开发框架。其特点在于简洁易用、基于生成代码的工具调用（核心抽象叫CodeAgent）以及与Hugging Face生态系统的集成。Smloagents与MCP的集成提供了一种直接的方式，可以为Agent添加复杂的功能，而无需为每个工具进行自定义编码。

【MCP集成】

以下代码演示了如何初始化一个Smloagent并将其连接到MCP Server：

代码块

```

1  from smolagents import ToolCollection, CodeAgent
2  from smolagents.agents import ToolCallingAgent
3  from smolagents import tool, LiteLLMModel
4  from mcp import StdioServerParameters
5  import os
6
7  model = LiteLLMModel(model_id="gpt-4o-mini")
8
9  server_parameters = StdioServerParameters(
10     command="npx",
11     args=["-y", "@mcptools/mcp-tavily"],
12     env={**os.environ},
13 )
14
15 with ToolCollection.from_mcp(server_parameters, trust_remote_code=True) as
    tool_collection:
16     agent = ToolCallingAgent(tools=[*tool_collection.tools], model=model)
17     response = agent.run("llama4.0发布了么? ")
18     print(response)

```

如果你需要使用SSE模式的MCP Server，只需要替换服务器配置参数为url即可。

七、Camel

【框架简介】



Camel是一个专注于创建能够进行复杂对话以解决任务的强大的多智能体构建框架。其独特之处在于使用AI Agent之间的角色扮演和交互协作来完成任任务，并内置了多种角色的Agent抽象及大量组件，Camel也可以用来开发RAG应用。现在这些Agent也可以通过MCP Server得到增强。

Camel还提供了一个将Camel中创建的工具集发布成MCP Server的功能。

【MCP集成】

你可以参考如下方式将基于Camel的Agent与MCP Server做集成：

代码块

```

1  import asyncio
2  from mcp.types import CallToolResult
3  from camel.toolkits.mcp_toolkit import MCPToolkit, MCPClient

```



```

4  import os
5  from camel.agents import ChatAgent
6
7  async def run_example():
8
9      mcp_client = MCPClient(
10          command_or_url="npx",
11          args=["-y", "@mcptools/mcp-tavily"],
12          env={**os.environ}
13      )
14      await mcp_client.connect()
15      mcp_toolkit = MCPToolkit(servers=[mcp_client])
16      tools = mcp_toolkit.get_tools()
17
18      try:
19          agent = ChatAgent(system_message='根据任务描述，使用网页搜索工具获取信息。',
20                          tools=tools)
21          response = await agent.astept("llama4.0发布了么? ")
22          print("Response:", response.msgs[0].content)
23      except Exception as e:
24          print(f"Error during agent execution: {e}")
25      finally:
26          # 确保在任何情况下都会断开连接
27          await mcp_client.disconnect()
28
29  if __name__ == "__main__":
30      asyncio.run(run_example())

```

如果需要连接SSE的远程Server，替换这里的MCPClient中的输入参数为url即可。

八、CrewAI

【框架简介】



CrewAI是一个用于编排自主AI智能体像团队一样协作完成复杂任务的多智能系统开发框架。其独特之处在于其“角色扮演”的设计，专注于创建具有特定角色和职责的结构化Agent团队（称为Crew）；最新的Flow功能可用于创建更可靠的Agentic Workflow。

【MCP集成】

目前官方的MCP集成正在紧锣密鼓的完善，还没有正式发布，暂时你可以借助一个第三方适配器进行：

代码块

```
1  import os
2  from crewai import Agent, Crew, Task # type: ignore
3  from mcp import StdioServerParameters
4  from mcpadapt.core import MCPAdapt
5  from mcpadapt.crewai_adapter import CrewAIAdapter
6
7  with MCPAdapt(
8      StdioServerParameters(
9          command="npx",
10         args=["-y", "@mcp-tools/mcp-tavily"],
11         env={**os.environ}
12     ),
13     CrewAIAdapter(),
14 ) as tools:
15     print(f"Tools: {tools}")
16     agent = Agent(
17         role="MyAgent", goal="根据任务描述，使用网页搜索工具获取信息。", backstory="你
是一个中文搜索助手",
18         tools=tools, llm='gpt-4o-mini',
19     )
20
21     # Create a task
22     task = Task(
23         description="llama4.0的最新消息", agent=agent, expected_output="消息列表")
24
25     task.execute_sync()
```

官方的MCP适配器的进展可以参考其Github开源项目的PR（MCP servers tool support in CrewAI）。

以上为大家盘点了8个常见的AI Agent开发框架及其对MCP的支持。由于MCP诞生不久且处于不断完善中，这些框架对MCP的适配也在不断迭代，请及时参考你所使用的开发框架的最新参考文档，了解最新变化。